

Scalabilité

Scalabilité verticale

Principe :

- À un instant t : une unité de calcul
- À $t+1$: la **même unité**, mais **plus puissante**

Idée clé : on augmente la capacité d'une **seule entité**.

Exemples :

- Ajouter de la RAM
- Ajouter des CPU
- Passer à un serveur plus puissant

Limites :

- Coût élevé
 - Limite physique du matériel
-

Scalabilité horizontale

Principe :

- À un instant t : une unité de calcul
- À $t+1$: **plusieurs unités de calcul**

Idée clé : on **multiplie les entités**.

Avantages :

- Moins cher que la verticale
- Plus résilient aux pannes
- Meilleure montée en charge

Exemple :

- Ajouter plusieurs serveurs plutôt qu'un seul très puissant
-

Pourquoi d'autres outils que les SGBD classiques ?

Big Data

Objectif :

- Traiter de **très gros volumes de données**
 - Données **structurées, semi-structurées ou non structurées**
-

NoSQL (Not Only SQL)

Objectif :

- Répondre aux besoins de **scalabilité horizontale**
 - Gérer des données massives et hétérogènes
-

3 Théorème CAP (Brewer)

Un système NoSQL **ne peut garantir que 2 propriétés sur 3.**

A – Availability (Disponibilité)

- Accès aux données **à tout moment**

P – Partition Tolerance (Tolérance au partitionnement)

- Le système continue de fonctionner **même si un nœud tombe**

C – Consistency (Cohérence)

- Les données sont **identiques et justes sur tous les nœuds**

 **Priorité typique NoSQL : $A > P > C$**

4 Types de bases de données NoSQL

Base orientée Clé–Valeur

Principe :

- Une **clé unique** associée à une **valeur** (entier, texte, XML...)

Paramètres de cohérence :

- **n** : facteur de réplication
- **R** : quorum de lecture
- **W** : quorum d'écriture

 **Cohérence forte si :**

$$R + W > N$$

 Plus de cohérence = lectures/écritures plus lentes

Base orientée Document

Principe :

- Stockage sous forme de **documents JSON / BSON / XML**
- Chaque document peut avoir une structure différente

Avantage clé :

- Grande **flexibilité du schéma**
-

Base orientée Colonnes (Wide-Column)

Principe :

- Données organisées **par colonnes**

Types :

- Colonnes statiques
- Colonnes dynamiques

Spécificité Cassandra :

- Une valeur absente = **la colonne n'existe pas** (pas de NULL)

Super-colonnes :

- Groupement de colonnes
-

Base orientée Graphe

Principe :

- Données sous forme de **nœuds** et **relations (arêtes)**

Utilisation typique :

- Réseaux sociaux
 - Relations complexes
-

5 Big Data – Les V

3V fondamentaux

- **Volume** : quantité de données
- **Vélocité** : vitesse de production
- **Variété** : formats multiples

4^e V

- **Véracité** : données fiables, utiles et de valeur
-

6 Correspondance NoSQL ↔ Domaines métiers

Orientation Domaine métier Explication

Clé–Valeur E-commerce Client = clé, panier = valeur

Document Catalogue produit Chaque produit a sa propre structure

Colonnes Catalogue massif Lecture rapide par colonnes

Orientation Domaine métier Explication

Graphe Réseaux sociaux Relations complexes entre utilisateurs

7 Comparatif des bases NoSQL

Clé-Valeur

- **SGBD** : Redis, DynamoDB, Riak
 - **Cas d'usage** : sessions, cache, paniers
 -  Ultra rapide, scalable
 -  Pas de requêtes complexes
-

Document

- **SGBD** : MongoDB, CouchDB
 - **Cas d'usage** : catalogues, profils
 -  Schéma flexible, requêtes riches
 -  Relations complexes difficiles
-

Colonnes

- **SGBD** : Cassandra, HBase
 - **Cas d'usage** : Big Data, logs, banques
 -  Très scalable, analytique
 -  Modélisation complexe
-

Graphes

- **SGBD** : Neo4j, OrientDB
 - **Cas d'usage** : réseaux sociaux, fraude
 -  Relations naturelles et rapides
 -  Peu adapté aux données tabulaires massives
-

8 À retenir absolument

- **Clé-Valeur** → rapidité, simplicité
- **Document** → flexibilité
- **Colonnes** → gros volumes et analyses
- **Graphes** → relations complexes